CITS5501 Project 2026 — Phase 1 Report

Group Number — 14 | Group Name — Scrapers

Group Members

Student ID	Student Name	GitHub ID
23904324	Adam Ahmadavi	**aradavi63**
24231774	Nikhil Chadha	**nikhilchadha28**
25039111	Nandhukrishna Gujjalapudi	**nandhugnk**
24771683	Cong Lu	**cong53077-spec**
24535749	Syeda Mehnaz Monsur	**Mehnaz-M**

GitHub Repository: https://github.com/aradavi63/CITS5501-G14

Task 1: Team Organisation

Collaboration & Workflow Plan

Team Communication and Responsibilities

Meetings: The group will have weekly face-to-face meetings on campus for approximately 30–60 minutes. These meetings will allow us to discuss what each member has completed, identify any issues, and decide what needs to be done the following week to advance the project.

Online Correspondence: The team will use Microsoft Teams for online communication, as all members already have access through their student accounts. Teams will be used for instant messaging, sharing updates, and video calls when a member is unable to attend a face-to-face meeting.

Accountability: GitHub Issues and GitHub Projects will be used to assign tasks and deadlines to individual members and track their progress. At the end of each meeting, we will also post a summary in the Microsoft Teams group chat outlining the main decisions and assigned tasks. This will help ensure that responsibilities are clearly recorded and visible to everyone.

Version Control Strategy

Hosting Service: We will use GitHub to host the project repository. All team members are comfortable with the basic features of Git and GitHub, including creating branches, committing changes, opening pull requests, reviewing code, and merging branches. Using a platform the team already understands will allow us to focus more time on developing and testing the project.

Member Contributions: Individual contributions will be visible through GitHub's commit history, pull requests, and Issues. Tasks will be assigned via GitHub Issues to ensure responsibilities are clear.

Each member will work on a separate feature branch when making significant changes. This should reduce unnecessary conflicts between members. If a merge conflict occurs, the members involved will resolve the conflicting changes and verify that the merged version still works correctly before it is added to the main branch.

Branching Strategy: The team will use GitHub Flow. The `main` branch will contain the stable version of the project, while development work will be completed using descriptively named feature branches, such as:

```
feature/document-parser
feature/search-function
fix/http-error-handling
```

Once work on a feature branch is complete, a pull request will be created. The changes will be reviewed and tested before being merged into `main`.

Version Control Policies

We will use consistent branch and commit naming conventions and regularly review code before merging changes. Automated tests and other quality checks will also be used where possible to reduce the chance of faulty code being merged into the main branch.

Commit Message Conventions:

Commit messages will be short and descriptive and will use the imperative present tense where appropriate.

Examples:

```
chore: initialise project
feat: add document parser
test: add parser tests
fix: handle invalid document link
docs: update project report
```

Merge and revert commits will also clearly identify the relevant branch or change.

Merge Approval Process:

Our general merge process will be:

1. A developer completes work on a feature or fix branch.
2. A pull request is created.
3. At least one other group member reviews the changes.
4. Automated tests and required quality checks must pass.
5. Any identified issues are corrected.
6. The approved pull request is merged into `main`.

Resource Risk Management

1. Resource Risk

Risk: Team member illness, missed deadlines, or conflicting schedules

Impact: High

Mitigation Strategy: Important tasks will not depend entirely on one person. More than one member should have enough context to continue important work if necessary. Weekly meetings will also be used to review workload and redistribute tasks where required.

2. Operational Risk
Risk: GitHub outage, local data loss, or synchronisation problems

Impact: Medium

Mitigation Strategy: Members will maintain local copies of the repository and commit and push changes regularly. Important project work will remain under version control so that previous versions can be recovered.

3. Technical Risk
Risk: Security or input-validation problems

Impact: High

Mitigation Strategy: We will follow secure coding practices, validate external input where appropriate, and use Java-compatible static-analysis tools such as Checkstyle and SpotBugs to identify potential quality issues.

4. Technical Risk
Risk: Missing dependencies, incompatible configurations, or development-environment differences

Impact: Medium

Mitigation Strategy: Maven will be used to manage project dependencies and builds. The team will use a consistent Java version and test the build process early to identify configuration problems.

5. Operational Risk
Risk: Merge conflicts or inconsistent code quality

Impact: High

Mitigation Strategy: New work will normally be completed on separate feature branches. Pull requests will be reviewed before merging, and automated tests and quality checks should pass before changes are added to main.

Task 2: Group Git Repository and Task Tracking

A private GitHub repository has been created for the project. All group members will have access to the repository and will be able to contribute through branches, commits, Issues, and pull requests.

Repository: https://github.com/aradavi63/CITS5501-G14

Task allocation and completion will be tracked using GitHub Issues and GitHub Projects.

Task 3: Report

Planning and Organisation

1. Where is your group's Collaboration & Workflow Plan maintained?
Our Collaboration & Workflow Plan is maintained in the group's GitHub repository. Keeping the plan in the same repository as the project's code and report gives the team one central location for project information and ensures that every member has access to the current version.

Maintaining the plan in GitHub also gives us a version history, so changes to responsibilities, workflow rules, deadlines, or development practices can be tracked rather than being lost across messages or separate documents. Important decisions can still be discussed during our weekly meetings or through Microsoft Teams, but the GitHub repository will act as the main source of truth for the documented workflow.

2. What tool or approach are you using to track assigned tasks and completion? Why did you choose it? Has anyone in the group used it before?
We are using GitHub Issues together with GitHub Projects to track tasks, ownership, deadlines, and completion. Issues allow individual pieces of work to be clearly described and assigned to specific group members, while GitHub Projects gives us a higher-level view of work that is planned, in progress, or completed.

We chose these tools because they are integrated directly into the same GitHub repository used for our source code. This avoids having to maintain a separate task-management system and allows issues to be connected to branches, commits, and pull requests. It should also make individual responsibilities and project progress easier to see throughout the semester.

Everyone in the group has previously used GitHub Issues, although not everyone has used GitHub Projects. The additional Project functionality is relatively straightforward and can be learned while we use it. At the end of weekly meetings, we will also summarise important decisions and assigned work in the Microsoft Teams group chat so that responsibilities are recorded in both the task tracker and our main communication channel.

Language and Tooling

3. Which language has your group chosen?
Our group has chosen **Java**. All members have at least some previous exposure to Java, including through the current unit's laboratory work, although individual levels of experience differ. If a

member is less confident with a particular part of Java, we can use peer code review, documentation, and discussion during meetings to help bring everyone's understanding to a similar level.

Our three most important factors, ranked in order, are:

1. **Familiarity with the language**
2. **Team members' experience**
3. **Library and tooling support**

Familiarity was the most important factor because the project already involves several new challenges, including web scraping, automated testing, and continuous integration. Choosing a language the group already understands reduces the amount of unrelated learning required before implementation can begin.

Team experience was similarly important because members need to understand and review one another's code rather than having isolated sections that only one person can maintain. Java also has mature libraries and development tools that should support HTTP communication, document processing, testing, and dependency management.

Java's static type system and compile-time checking should help identify some mistakes before the application is run. It also integrates well with tools we intend to use, including JUnit 5 for testing and Maven for builds and dependency management. Its structured approach should also help us maintain consistent interfaces between components being developed by different group members.

There are still disadvantages. Java can require more boilerplate than Python, and tasks such as web scraping, text processing, and handling several document formats may require more setup or external libraries. Differences in Java experience between members could also produce inconsistent code unless we maintain code-review and formatting standards.

If we had instead chosen Python, rapid prototyping and some web-scraping or text-processing tasks might be easier because of Python's concise syntax and extensive libraries. However, the team would need to establish a different dependency-management, testing, and type-checking workflow, and the weaker compile-time checking would place more importance on our test suite and static-analysis tools.

4. What build/dependency management tool will you use? Has anyone in the group used it before?

We intend to use **Maven** as our build and dependency-management tool. Maven is not entirely new to the group and was selected partly because members already have some familiarity with it.

Maven gives us a standard way to define the project's dependencies and build configuration in a `pom.xml` file rather than requiring every member to install and configure libraries manually. This is particularly useful for a group project because everyone can build and test the project using the same dependency versions and configuration.

It also provides a conventional project structure and integrates with tools we intend to use, particularly JUnit 5. Maven can run the test suite as part of the build process, which should also make it easier to integrate our tests with GitHub Actions later in the project.

A consistent Maven build should reduce problems where code works on one group member's computer but fails on another due to missing libraries or different local configurations. Although some Maven configuration may be unfamiliar, using a standard build tool gives the project a repeatable build process and should make dependency management easier as the web scraper begins to require additional libraries.

Technical Risks

5. What parts of a web scraper do you expect could most easily fail, and how?
Several parts of the web scraper could fail:

Connection to the website: The connection may fail if internet access is lost, the website's server becomes temporarily unavailable, or the server takes too long to respond. As a result, the scraper may fail to retrieve information or a document download may be incomplete.

Reading webpage markup: This part may fail if the webpage contains malformed or unexpected HTML. Even if the scraper successfully opens the webpage, it may not recognise the required information if the structure is different from what the program expects.

Processing documents: The scraper will need to work with different document formats such as PDF, Word, and RTF. Unexpected or malformed documents could cause extraction or processing failures.

Processing server responses: The scraper must correctly recognise responses such as server errors, access-denied pages, redirects, or missing resources. Otherwise, it could incorrectly process an error response as valid content.

Managing request frequency: If the scraper sends too many requests to the server within a short period, the server may limit or reject requests. This could result in the scraper missing webpages or policy documents.

6. Which external factors could make your software stop working after it has been completed?
Several external factors could cause the software to stop working after completion.

Changes in website structure: The layout or HTML structure of the website may change after the scraper has been completed. Document links or other elements may also move, causing the scraper to no longer find the required information.

Changes to website access: The website could introduce authentication, CAPTCHA, additional security restrictions, or stronger rate limiting. These changes could prevent automated requests from working as expected.

New document formats: The website may begin providing documents in a format that the software does not support. The scraper may still retrieve the document but may be unable to extract its contents.

Website or server unavailability: The website may temporarily become unavailable because of maintenance, network problems, or server failure. Since the scraper depends on the external website, it may not be able to retrieve information during these periods.

7. Which components of your project do you think will be easiest to test? Which do you expect will be hardest?
Components easiest to test:

Searching component: This should be relatively easy to test because we can create documents with known content, search for particular words or phrases, and compare the returned results with the expected results.

Identifying document formats: We can provide test data containing known document formats and check whether the software correctly identifies each format.

Components hardest to test:

Communication with the real website: This may be difficult to test consistently because it depends on factors outside our control, including internet connectivity, server availability, response times, and request limits.

Complete software process: End-to-end testing may also be difficult because several components are involved. A failure in downloading, parsing, storing, or searching documents could affect the final result.

8. What assumptions is your software likely to make about the website it scrapes? How could those assumptions be violated?
Our software is likely to make several assumptions.

Consistent website structure: The scraper may assume that the website follows a consistent structure. This assumption could be violated if the website developers change its HTML structure or the way document links are presented.

Supported document formats: The software may assume that policy documents are provided in supported formats such as PDF, Word, or RTF. This assumption could be violated if the website introduces unsupported formats, such as ZIP archives or images.

Valid links and responses: The scraper may assume that document links lead to valid resources and that successful server responses contain the expected content. Broken links, redirects, access-denied responses, or malformed documents could violate this assumption.

Website availability: The software may assume that the website is reachable when scraping begins. Temporary server outages, network problems, or request restrictions could violate this assumption.

Team Practices

9. Suppose a serious bug is discovered two days before the deadline. How would your team investigate and respond?

First, the bug will be reported to the whole group through our communication channel and recorded as an issue. We will then reproduce the problem and assess its severity, particularly whether it affects a core requirement or only a minor feature.

A team member will be assigned to investigate the cause and, where possible, create a test that reproduces the failure. The fix will be developed on a separate branch and reviewed before merging.

After the bug is fixed, we will run the relevant tests as well as regression tests to check that the change has not broken existing functionality. If the bug cannot be safely resolved before the deadline, we will clearly document the known issue rather than introduce a rushed change that could create further problems.

10. What will convince your team that a feature is "finished"?

A feature will be considered finished when:

- It behaves according to the agreed requirements.
- Relevant automated tests pass.
- Important edge cases and error conditions have been considered.
- At least one other team member has reviewed the code.
- Required static-analysis or quality checks pass.
- The feature has been successfully integrated without breaking existing functionality.

11. If two group members disagree about whether software is "working correctly", how will you resolve that disagreement?

We will first compare the software's behaviour with the project specification and any relevant clarifications provided by the teaching staff. Rather than relying only on personal opinions or a majority vote, we will try to reproduce the disputed behaviour and define a test case with a clear expected result.

If the test demonstrates that the software does not meet the specified behaviour, the implementation will be corrected. If the specification remains ambiguous and the group cannot reach an evidence-based conclusion, we will ask our facilitator or post a clarification question through the appropriate unit discussion forum.

Code Quality

12. Is your team familiar with any static analysis/linting tools? Do you plan to use any?
Our group does not currently have extensive experience with Java static-analysis tools such as Checkstyle, SpotBugs, or PMD. However, we plan to use **Checkstyle and SpotBugs** during development.

Checkstyle: Checkstyle will help ensure that code written by different team members follows consistent formatting and coding conventions.

SpotBugs: SpotBugs will help identify common programming mistakes and potential defects through static analysis.

We may also consider PMD if additional static-analysis checks are useful later in development, but our initial focus will be on setting up and consistently using Checkstyle and SpotBugs.

13. Suppose a group member's contribution doesn't pass your linting or type-checking checks. What will be the consequences?
Merges will be blocked until the contribution passes the required checks. If linting, compilation, static analysis, or other required quality checks fail, the contributor will need to investigate and correct the problems before the code can be merged into the `main` branch.

This will make the checks part of our quality process rather than treating them only as advisory warnings.

Testing Framework

14. What test framework will you use? Has the group used it before, or only heard of it?
We will use **JUnit 5** as our testing framework. Everyone in the group has already used JUnit 5 in the unit laboratories, so we are familiar with the basic process of writing and running tests.

JUnit 5 also integrates well with Maven, which should allow us to run the test suite consistently on different group members' machines and through our continuous-integration workflow.

CI / Automation

15. Will you set up continuous integration or run tests manually before submission? Have any members of your group used continuous integration before?
We plan to use **GitHub Actions** for continuous integration so that our test suite and relevant quality checks can run automatically when changes are pushed or a pull request is created.

This will allow us to identify problems throughout development rather than waiting until immediately before submission. If a test or required check fails, we can investigate whether the problem is in the implementation or the test itself before the changes are merged.

None of our group members have significant previous experience with continuous integration or GitHub Actions. However, we plan to learn and set it up as part of the project.

Testing Mindset

16. Suppose your scraper is finished and submitted. Now imagine another student is trying to break your software. What might they try first?

They might first provide invalid or broken links, pages containing unexpected markup, or unsupported file types to see whether the scraper crashes or handles the problem safely.

They could also try pages with missing content, malformed documents, duplicate links, very large files, or unusual server responses. Another possibility would be triggering many requests within a short period to see how the software behaves when the website applies rate limits or starts rejecting requests.

These cases would help reveal whether the program handles unexpected input and external failures gracefully rather than assuming that every request and document will be valid.

17. How will you know that your software is producing the correct output, rather than merely some output?

We will compare the scraper's output with the actual content available on the website. We will test it using known examples where we already know what the expected result should be.

We will also check different document types and edge cases to make sure the program is retrieving and processing the correct information, rather than simply returning some output.

18. What evidence would increase your confidence that your software is reliable?

Our confidence would increase if the software consistently passes a range of automated tests, including normal cases, edge cases, and expected failure scenarios.

We would also test the scraper multiple times with different pages and document formats. Successful tests for situations such as broken links, unexpected responses, malformed documents, and temporary server issues would give us more confidence that the software is reliable.

19. At this stage of the project, what do you think will be the biggest challenge in producing reliable software?

We think the biggest challenge will be handling unexpected behaviour from the website. Since a web scraper depends on an external website, changes in page structure, broken links, malformed content, server errors, or request limits could cause the program to fail.

Designing the software to handle these situations properly while still producing correct results will probably be our main challenge.

Process and Standards

20. What will be your code review process?
We plan to use a peer-review process. Before significant changes are merged into the `main` branch, at least one group member other than the author will review the code.

The reviewer will check whether the code is understandable, follows our agreed coding practices, satisfies the relevant requirements, and does not introduce obvious problems. Relevant automated tests and quality checks should also pass before merging.

For very small changes, such as minor documentation corrections, the review may be more informal. However, new features, significant changes, and bug fixes should be reviewed through a pull request before being merged.

Repository Information

Private GitHub Repository: https://github.com/aradavi63/CITS5501-G14

Task Tracking
GitHub Projects Board: https://github.com/users/aradavi63/projects/3

End of Phase 1 Report